PYTHON CLASS AND OBJECTS

OBJECT

- Python is an object-oriented programming language. So, its main focus is on objects instead on functions
- Almost everything in python is an **object**, with its properties and methods.
- In oops, object is a collection of data (variables) and methods (functions) that act on those data
- Object is also called an instance of a class
- Process of creating an object is called as instantiation.

CLASS

- Class is a set of objects that shares common structures and common behaviors. (Container for variables and methods)
  - Here structure points to variables
  - Behavior points to methods.
- Class is a user defined data type that includes a set of variables and methods.
- In python, a class is defined by using a reserved keyword **class**.

Syntax

```
class <class-name>:
        # variable section
        # method section
```

Example

```
class welcome:
        def show(self):
                print("Hello World")
```

NOTE

- It is an important to note that, **a colon (:) operator must be marked at end of the class and function declarations.**

Where,

class → reserved word which is used to define the class

def → reserved word, which is used to define the function

print → built-in function, which is used to print the data on output console.

SELF PARAMETER

- It is a built-in parameter which is a reference to the current class. (self refers to current object of a class)
- It represents the instance of a class. It is used to access instance members of a class like instance variables
- It does not have any name, so we can name it whatever we like, **but it must be placed as a first parameter of any instance function** in the class.

Example

```
class welcome:
    def disp(sel):
        print("Hello World")
```

OBJECT CREATION IN PYTHON

- An object is **an instance of a class** and it will be stored on heap memory
- Process of creating an object is called as instantiating an object (instantiation).
- It is **a reference type** (not value type)
- It is an important to note that, **object occupies memory only for variables not methods**. Because methods are independent of all objects of a particular class.

Syntax

```
<object-name> =<class-name>(arg1, arg2, … argn)
```

Where,

arg_1, arg_2, …arg_n → list of actual parameters

Example

```
obj=Test()
```

Where,

- obj is the name of the object
- Test is the name of the class.

EMPTY CLASS

- In python, the empty class is created with help of pass statement
- Pass is a reserved word, which is used to create the empty statements in python

Example

```
class Test:
    pass
```

TYPES OF VARIABLES – PYTHON OOPS

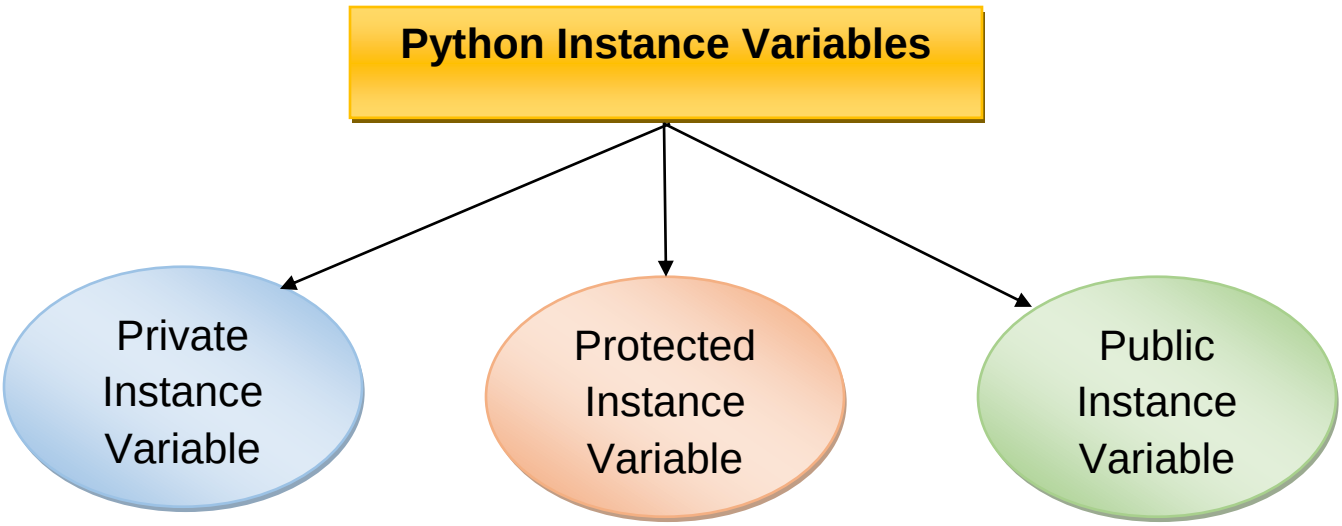
- Like java, python supports three types of variables. They are
    1. Instance variable
    2. Static variable
    3. Local variable.

1. INSTANCE VARIABLE

- If a variable is placed either inside a function or constructor **with self keyword**, then it is called as instance variable
- Unlike c++/ java, the python instance variable is placed inside a function or constructor
- In python, the instance variables are defined with help of access modifiers
- Instance variables are specified by preceding the name with the self reference

PYTHON ACCESS MODIFIERS

- Unlike c++/java, python does **not directly support access modifiers.** But it indirectly supports access modifiers using operator symbols.
- In python, we can define three types of instance variables using access modifiers. They are
    1. Private instance variable
    2. Protected instance variable
    3. Public instance variable
- It supports access modifiers with help of operator symbols like **single underscore** (_) and **double underscore** (__)
- In Python, all member variables and methods are **public by default**.



1.1 PRIVATE VARIABLE (PRIVATE INSTANCE VARIABLE)

- Private variable is accessible only within the same class.
- In python, the **double underscore (_)** is used to define the private variable along with self parameter
- It is not possible to access the private variable in outside of a class
- It is used for security purpose.

Syntax

```
self.__variable-name=value
```

Example

```
self.__k=73                    # private instance variable
```

NOTE

- It is an important to note that, unlike c++/ java, the python instance variable is placed either inside of a function or constructor.

I. EXAMPLE OF PRIVATE INSTANCE VARIABLE

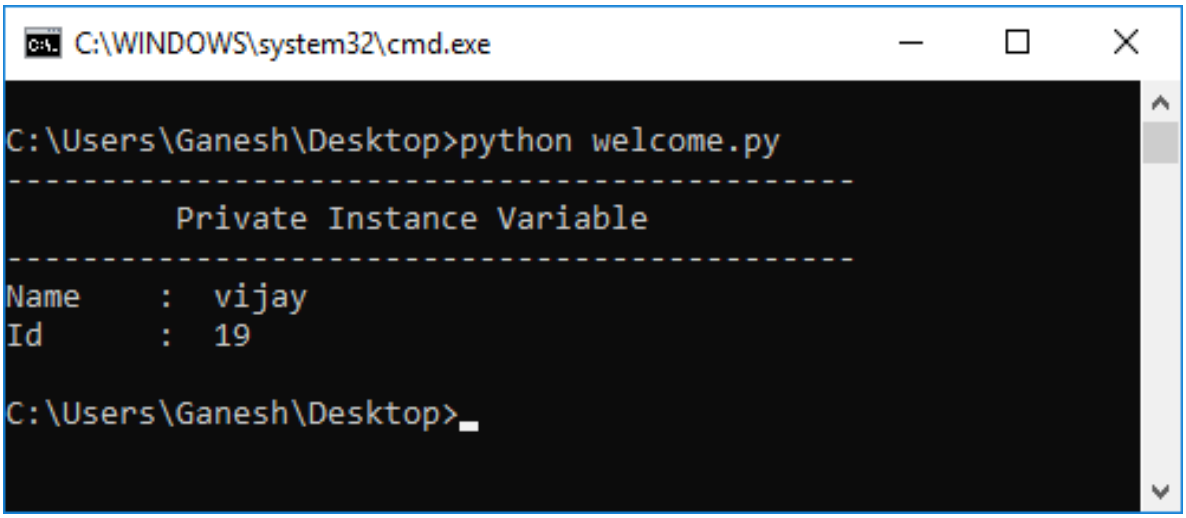
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Private Instance Variable")
print("-----------------------------------------")
class Hello:
    def disp(self):
        self.__id=19
        self.__name="vijay"
    def show(self):
        print("Name \t: ",self.__name)
        print("Id \t: ",self.__id)
# object creation
obj=Hello()
# calling instance methods
obj.disp()
obj.show()
```

Private instance variables

2. OUTPUT



1.2 PROTECTED VARIABLE (PROTECTED INSTANCE VARIABLE)

- Protected variable is accessible only within the same class and its subclasses
- In python, the single underscore (_) is used to define the protected variable along with self parameter
- Unlike c++ / java, it is possible to access the protected variable in outside of a class in python.

Syntax

```
self._variable-name=value
```

Example

```
self._place=”salem”        # protected instance variable
```

II. EXAMPLE OF PROTECTED INSTANCE VARIABLE

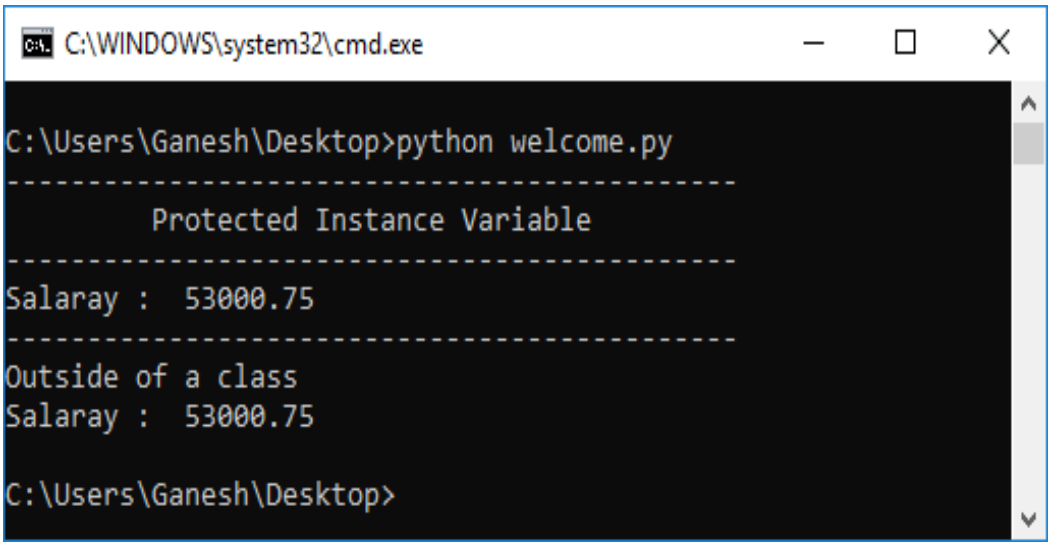
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Protected Instance Variable")
print("-----------------------------------------")
class Hello:
        def disp(self):
                self._salary=53000.75
                print("Salaray : ",self._salary)
# object creation
obj=Hello()
# calling instance method
obj.disp()
print("-----------------------------------------")
print("Outside of a class")
print("Salaray : ",obj._salary)
```

Protected instance variable

2. OUTPUT



1.3 PUBLIC VARIABLE (PUBLIC INSTANCE VARIABLE)

- Public variable is accessible anywhere in the programming (same class or outside of a class or outside of a package)
- In Python, all member variables and methods are **public by default**.
- So, no need to use single underscore (_) and double underscore (__) operator symbols in the declaration of public variable.

Syntax

```
self.variable-name=value
```

Example

```
self.salary=75400.95        # public instance variable
```

III. EXAMPLE OF PUBLIC INSTANCE VARIABLE

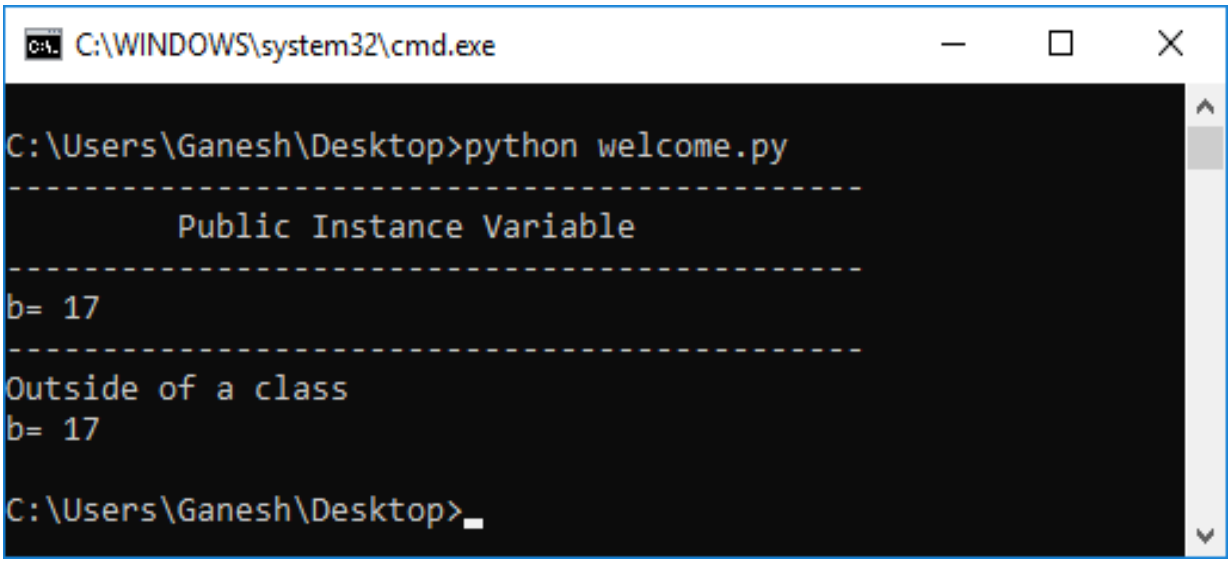
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Public Instance Variable")
print("-----------------------------------------")
class Hello:
        def disp(self):
                self.b=17
                print("b=",self.b)
# object creation
obj=Hello()
# calling instance method
obj.disp()
print("-----------------------------------------")
print("Outside of a class")
print("b=",obj.b)
```

Public instance variable

2. OUTPUT



2. LOCAL VARIABLE

- If a variable is placed either inside a function or constructor without self keyword, then it is called as local variable
- Unlike instance variables, it cannot be shared (accessed) in all functions. It can be accessed only in same function
- No need to use self keyword, to define the local variable in the class definition.

IV. EXAMPLE OF LOCAL VARIABLE

(welcome.py)

1. SOURCE CODE

```
print("-------------------------------------------")
print("\t Local Variable in a Class")
print("-------------------------------------------")
class Hello:
    def disp(self):
        n=93
        print("n=",n)
# object creation
obj=Hello()
# calling instance method using object
obj.disp()
```

Local variable

2. OUTPUT

```
C:\WINDOWS\system32\cmd.exe

C:\Users\Ganesh\Desktop>python welcome.py
-------------------------------------------
         Local Variable in a Class
-------------------------------------------
n= 93

C:\Users\Ganesh\Desktop>
```

3. STATIC VARIABLE

- If a variable is placed inside the class without any modifier **but not inside a method** that is called as static variable
- All variables defined on the class level in python are considered static
- It **purely depends on class**, so that it can be called by using the class name
- A static variable is a variable that has been allocated statically, meaning that its lifetime is the entire run of the program.
- Like instance variable, it be accessed in all the functions (Global scope).

Calling

- The static variable can be called or accessed by using the class name. Because it purely depends on class
- It is also possible to call static variable using object.

Syntax

```
<class-name>.<static-variable>
```

Example

```
print(Welcome.msg)
```

V. EXAMPLE OF STATIC VARIABLE

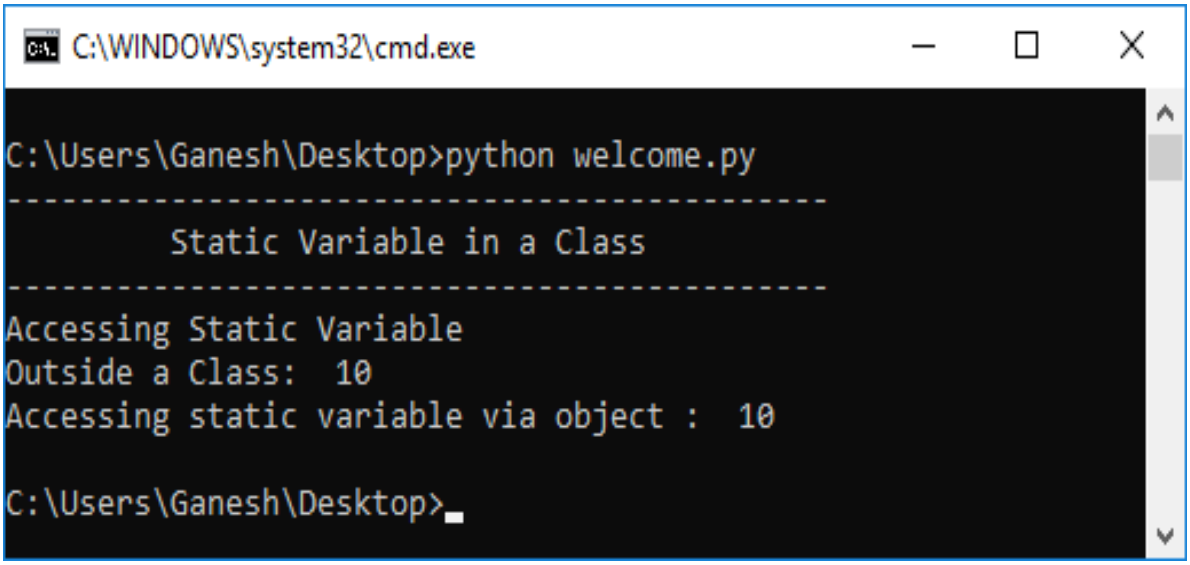
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Static Variable in a Class")
print("-----------------------------------------")
# class definition
class Hello:
        n=10
        def disp(self):
                print("Accessing Static Variable\nInside a Method: ", Hello.n)
print("Accessing Static Variable\nOutside a Class: ", Hello.n)
# object creation
obj=Hello()
print("Accessing static variable via object : ",obj.n)
```

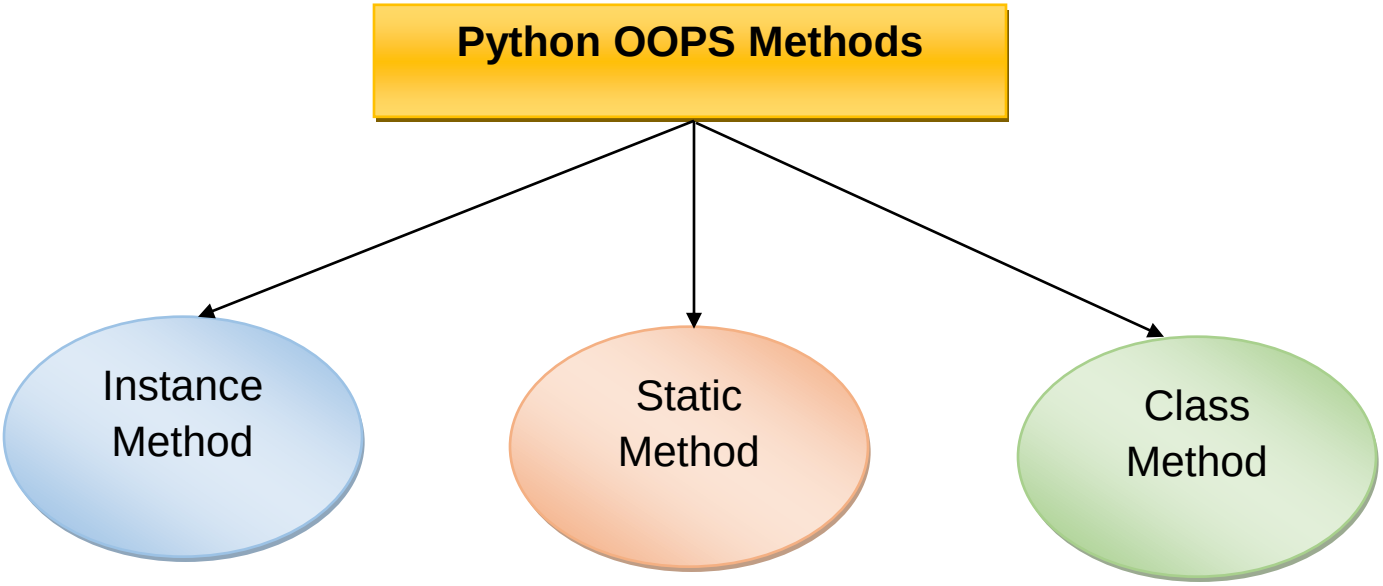
Static variable

2. OUTPUT



PYTHON OOPS METHODS

- Python supports three types of methods. They are
  1. Instance Method
  2. Static Method
  3. Class Method




```
class Test:
    def method(self):
        return 'instance method called', self

    @classmethod
    def classmethod(cls):
        return 'class method called', cls

    @staticmethod
    def staticmethod():
        return 'static method called'
```

PYTHON INSTANCE METHOD

- This is a regular method or instance method which depends on object
- It takes one parameter self, which points to an **instance of the current class**
- The self parameter is **mandatory section when defining a method inside the class**.

Syntax

```
def <function-name>(self, a1, a2, …an):
        # user code
```

Example

```
def disp(self):
        print("Good Morning")
```

Calling

- Instance method **purely depends on object**. So, it must be called by using object name along with dot operator.

Syntax

```
<object-name>.<function-name(a1, a2, ..an)>
```

Example

```
obj.info()
```

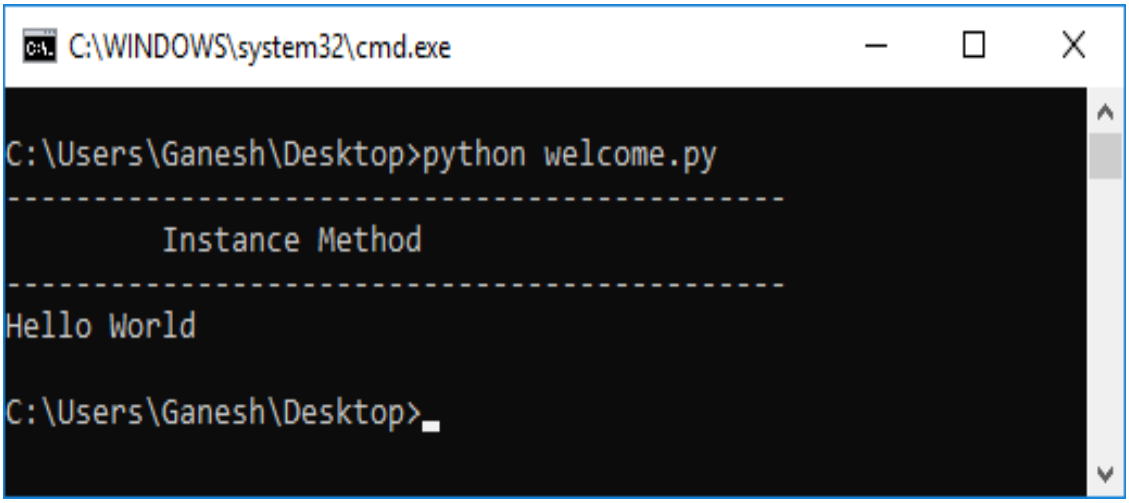
VI. EXAMPLE OF INSTANCE METHOD

(welcome.py)

1. SOURCE CODE

```
print("-------------------------------------------")
print("\t Instance Method")
print("-------------------------------------------")
# class definition
class Hello:
# instance method definition (public method)
    def info(self):          # note that first argument is self
        print("Hello World")
# object creation
obj=Hello()
# calling instance method using object
obj.info()
```

2. OUTPUT



PYTHON STATIC METHOD

- If a method is marked with @staticmethod decorator, then it is called as static method
- Unlike instance method, this method takes neither a self nor a cls parameter
- It depends on class and will not use object at all
- Static method can call other static and class methods.

Syntax

```
@staticmethod
def <function-name>(a1, a2, …an):
        # user code
```

Example

```
@staticmethod
def disp():
        print("Good Morning")
```

Calling Static Method

- Static method purely **depends on class**. So, it must be called by using class name along with dot operator.

Syntax

```
<class-name>.<function-name(a1, a2, ..an)>
```

Example

```
Test.info()
```

Where,

- Test is a current class and info() is a static method.

VII. EXAMPLE OF STATIC METHOD

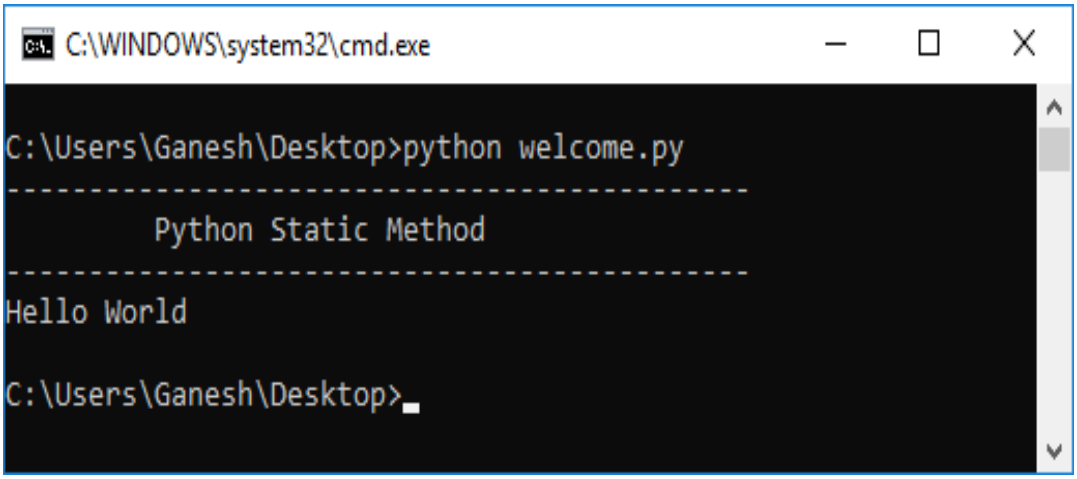
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Python Static Method")
print("-----------------------------------------")
# class definition
class Hello:
# static method definition
        @staticmethod
        def show():
                print("Hello World")
# calling static method using class name
Hello.show()
```

Static method

2. OUTPUT



NOTE

- No need to pass self parameter as a first parameter in the creation of static method.

PYTHON CLASS METHOD

- If a method is marked with @classmethod decorator, then it is called as class method
- Instead of accepting a self parameter, class method takes a cls parameter, that points to the current class (not the object)
- The cls parameter represents the class and it is mandatory section.

Syntax

```
@classmethod
def <function-name>(cls, a1, a2, …an):
        # user code
```

Example

```
@classmethod
def disp(cls):
        print("Good Morning")
```

Where,

- cls stands for class which represents the current class.

NOTE

- **cls** refers to the **class** whereas **self** refers to the **instance**.
- Using the cls keyword, we can call only the class members (static members), whereas using the self keyword, we can access both the instance and static members of the class.

VIII. EXAMPLE OF CLASS METHOD

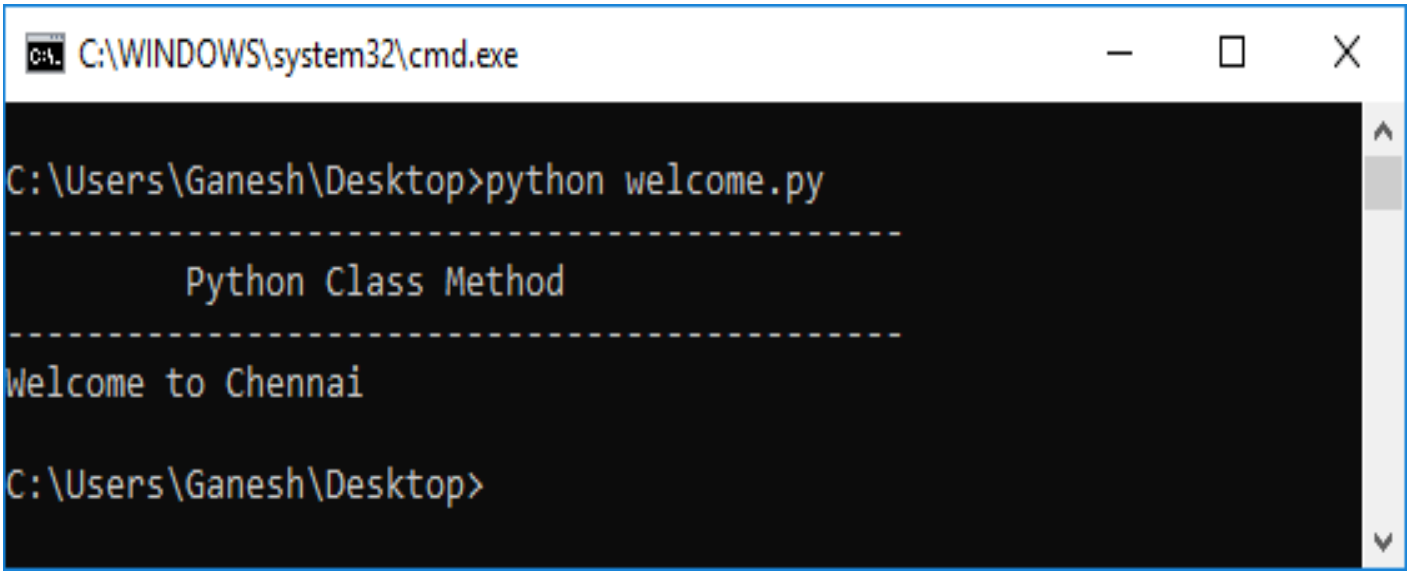
(welcome.py)

1. SOURCE CODE

```
print("-----------------------------------------")
print("\t Python Class Method")
print("-----------------------------------------")
# class definition
class Hello:
        @classmethod
        def info(cls):
                print("Welcome to Chennai")
# calling class method using class name
Hello.info()
```

Class method

2. OUTPUT



COMPARISON OF INSTANCE, STATIC AND CLASS METHODS

S.N	INSTANCE METHOD	CLASS METHOD	STATIC METHOD
1.	It depends on object	It depends on class	It depends to class and will not use object at all
2.	Regular method	Method is marked with **@classmethod** decorator	Method is marked with **@staticmethod** decorator
3.	It has self parameter as a first parameter	It has cls parameter as a first parameter	It doesn't have access to cls or self parameter (simple function with self argument)
4.	It can be accessed by using object	It can be called either using class or object	It can be accessed either using class or object